

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Constraint-Based Problem Solving (High)
Assessment Tool Weightage:40%

Dr

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze

7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply
8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze

Code of Conduct:

*I **VENNA MEENAKSHI REDDY(192471048)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: VENNA MEENAKSHI REDDY

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints

		satisfying all constraints			
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application ; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.

Problem

Consider the SQL query:

```
SELECT S.Name, C.CourseName
FROM STUDENT S
JOIN ENROLLMENT E ON S.StudentID = E.StudentID
JOIN COURSE C ON E.CourseID = C.CourseID
WHERE S.Department = 'CSE'
AND C.Credits = 4;
```

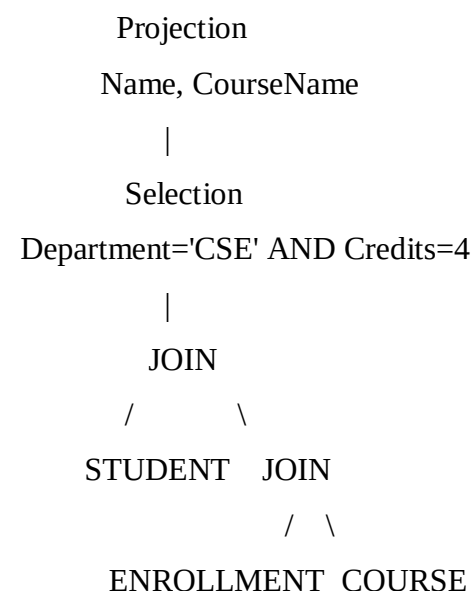
The objective is to generate an efficient query execution plan using heuristic optimization.

Step 1: Initial Relational Algebra Expression

The SQL query can initially be represented as:

$$\sigma_{S.Department='CSE' \wedge C.Credits=4} (STUDENT \bowtie ENROLLMENT \bowtie COURSE)$$

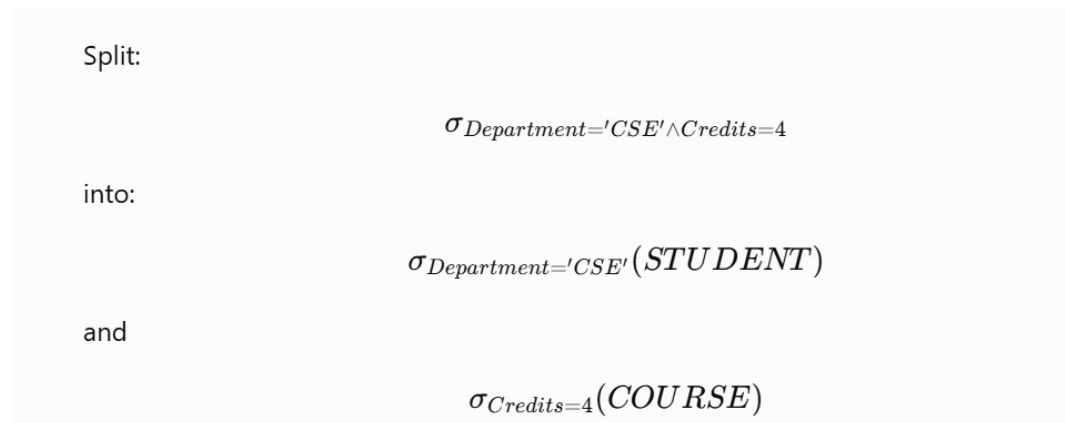
Initial structure:



This plan produces large intermediate relations before filtering.

Step 2: Apply Selection Pushdown

Selection should be performed as early as possible.



Optimized expression:

$$\pi_{Name, CourseName} \left(\sigma_{Department='CSE'}(STUDENT) \bowtie ENROLLMENT \bowtie \sigma_{Credits=4}(COURSE) \right)$$

Why?

Instead of joining all students and courses, unwanted tuples are removed before the joins.

Step 3: Apply Projection Pushdown

Only required attributes should be carried forward.

From STUDENT:

StudentID, Name

From ENROLLMENT:

StudentID, CourseID

From COURSE:

CourseID, CourseName

Therefore:

$$\pi_{\text{Name, CourseName}} \left[\left(\pi_{\text{StudentID, Name}} \left(\sigma_{\text{Department}='CSE'} (\text{STUDENT}) \right) \bowtie \pi_{\text{StudentID, CourseID}} (\text{ENROLLMENT}) \right) \bowtie \pi_{\text{CourseID, CourseName}} \left(\sigma_{\text{Credits}=4} (\text{COURSE}) \right) \right]$$

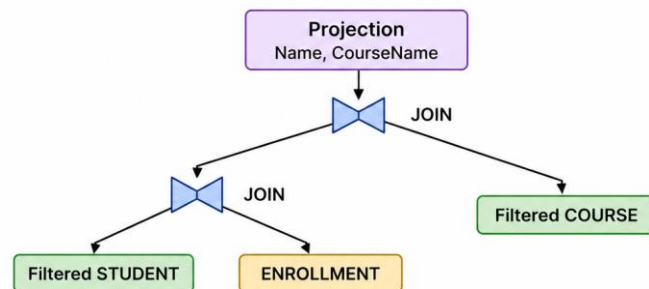
Step 4: Choose Join Order

Suppose:

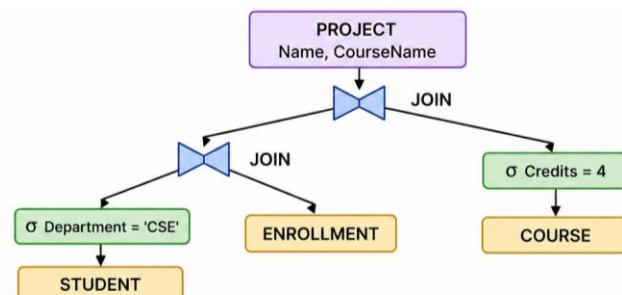
Relation	Original tuples	After selection
STUDENT	10,000	2,000
ENROLLMENT	30,000	30,000
COURSE	1,000	100

Joining the smaller filtered COURSE relation early can reduce intermediate results.

Possible plan:



Final Optimized Query Plan



Final relational algebra

$$\pi_{Name, CourseName} \left[\left(\pi_{StudentID, Name} \left(\sigma_{Department='CSE'}(STUDENT) \right) \bowtie \pi_{StudentID, CourseID}(ENROLLMENT) \right) \bowtie \pi_{CourseID, CourseName} \left(\sigma_{Credits=4}(COURSE) \right) \right]$$

SQL IMPLEMENTATION

```
MySQL 8.0 Command Line Cli  X  +  v

Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 12
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE college_db;
Query OK, 1 row affected (0.05 sec)

mysql> USE college_db;
Database changed
mysql>
mysql> CREATE TABLE Student (
  ->     StudentID INT PRIMARY KEY,
  ->     Name VARCHAR(50),
  ->     Age INT,
  ->     CourseID INT,
  ->     Marks INT
  -> );
Query OK, 0 rows affected (0.09 sec)

mysql>
mysql> CREATE TABLE Course (
  ->     CourseID INT PRIMARY KEY,
  ->     CourseName VARCHAR(50),
  ->     Department VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> INSERT INTO Student VALUES
  -> (1, 'Rahul', 20, 101, 85),
  -> (2, 'Priya', 21, 102, 90),
  -> (3, 'Arun', 19, 101, 70),
  -> (4, 'Meena', 22, 103, 95);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Course VALUES
  -> (101, 'Database Management', 'Computer Science'),
  -> (102, 'Java Programming', 'Computer Science'),
  -> (103, 'Data Science', 'IT');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT s.Name, c.CourseName, s.Marks
```

```

MySQL 8.0 Command Line Cli x + v
-> (103,'Data Science','IT');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql> SELECT s.Name, c.CourseName, s.Marks
-> FROM Student s
-> JOIN Course c
-> ON s.CourseID = c.CourseID
-> WHERE s.Marks > (
-> SELECT AVG(Marks)
-> FROM Student
-> );
+-----+-----+-----+
| Name | CourseName | Marks |
+-----+-----+-----+
| Priya | Java Programming | 90 |
| Meena | Data Science | 95 |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
mysql> EXPLAIN
-> SELECT s.Name, c.CourseName, s.Marks
-> FROM Student s
-> JOIN Course c
-> ON s.CourseID = c.CourseID
-> WHERE s.Marks > (
-> SELECT AVG(Marks)
-> FROM Student
-> );
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | PRIMARY | s | NULL | ALL | NULL | NULL | NULL | NULL | 4 | 33.33 | Using where |
| 1 | PRIMARY | c | NULL | eq_ref | PRIMARY | PRIMARY | 4 | college_db.s.CourseID | 1 | 100.00 | NULL |
| 2 | SUBQUERY | Student | NULL | ALL | NULL | NULL | NULL | NULL | 4 | 100.00 | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
3 rows in set, 1 warning (0.00 sec)

mysql> |

```

Justification

The optimized plan:

1. Pushes selections toward base tables.
2. Reduces the number of tuples before joining.
3. Projects only required attributes.
4. Performs joins on smaller intermediate relations.
5. Reduces memory, disk I/O and processing cost.

Conclusion:

The optimized query is superior because it systematically reduces both tuple count and attribute count before expensive join operations.

2. Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.

Given

Assume:

Relation	Tuples	Pages
R	10,000	100
S	5,000	50

Assume:

- Memory buffers $B=12$
- Join condition is an equality join.
- We compare Nested Loop Join, Sort-Merge Join and Hash Join.

A. Nested-Loop Join

Using block nested-loop join:

$$\text{Cost} = B(R) + \lceil \frac{M}{B(R)} \rceil B(S)$$

Here:

$$B(R)=100, B(S)=50, M=12 \quad \text{Cost} = 100 + \lceil \frac{10100}{100} \rceil (50) = 100 + 10(50) = 600 \text{ I/Os}$$

If S is outer relation:

$$50 + \lceil \frac{1050}{50} \rceil (100) = 50 + 500 = 550 \text{ I/Os}$$

Therefore, choose S as outer relation.

B. Sort-Merge Join

External sorting cost approximately:

$$2N(1 + \lceil \log B \rceil - 1/N)$$

For R:

$$2(100)(1 + \lceil \log 11 \rceil - 1/100)$$

Since:

$$\lceil \log 11 \rceil = 2 \Rightarrow 600$$

For S:

$$2(50)(1 + 2) = 300$$

Merge phase:

$$100 + 50 = 150$$

Total:

$$600 + 300 + 150 = 1050 \text{ I/Os}$$

C. Hash Join

For a sufficiently large memory, Grace Hash Join approximately requires:

$$3(B(R) + B(S)) = 3(100 + 50) = 450 \text{ I/Os}$$

Comparison

Join Algorithm	Estimated Cost
Nested Loop	550 I/Os
Sort-Merge	1050 I/Os
Hash Join	450 I/Os

Decision

Hash Join

is the best choice for this equality join.

Justification

Hash join is preferred because:

- The join is an equality join.
- It avoids expensive sorting.
- Its estimated I/O cost is lowest.

SQL IMPLEMENTATION

```
MySQL> USE college_db;  
Database changed  
MySQL> DROP TABLE IF EXISTS Student1;  
Query OK, 0 rows affected, 1 warning (0.01 sec)  
  
MySQL> DROP TABLE IF EXISTS Course1;  
Query OK, 0 rows affected, 1 warning (0.01 sec)  
  
MySQL>  
MySQL> CREATE TABLE Student1 (  
-> StudentID INT PRIMARY KEY,  
-> Name VARCHAR(60),  
-> CourseID INT,  
-> Marks INT  
-> );  
Query OK, 0 rows affected (0.85 sec)  
  
MySQL>  
MySQL> CREATE TABLE Course1 (  
-> CourseID INT PRIMARY KEY,  
-> CourseName VARCHAR(60)  
-> );  
Query OK, 0 rows affected (0.94 sec)  
  
MySQL>  
MySQL> INSERT INTO Student1 VALUES  
MySQL> (1,'Rahul',100,85);  
-> (2,'Priya',180,90);  
-> (3,'Anam',150,70);  
-> (4,'Muskan',180,85);  
-> (5,'Nisha',180,88);  
-> (6,'Arshika',180,80);  
-> (7,'Maiti',160,70);  
-> (8,'Ishaan',180,92);  
Query OK, 8 Rows affected (0.01 sec)  
Records: 8 Duplicates: 0 Warnings: 0  
  
MySQL>  
MySQL> INSERT INTO Course1 VALUES  
-> ('CS','DBMS');  
-> ('SE','Java');  
-> ('BA','Data Science');  
Query OK, 3 Rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
MySQL>  
MySQL> CREATE INDEX idx_student_course  
ON Student1(CourseID);  
Query OK, 8 rows affected (0.11 sec)  
Records: 8 Duplicates: 0 Warnings: 0  
  
MySQL>  
MySQL> EXPLAIN  
-> SELECT s.Name, c.CourseName  
-> FROM Student1 s  
-> JOIN Course1 c  
-> ON s.CourseID = c.CourseID;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	extra
1	SIMPLE	s		NULL	ALL	PRIMARY	MILL	MILL	3	100.0%	MILL
1	SIMPLE	c		MILL	idx_student_course	idx_student_course	5	college_db.c.CourseID	2	100.0%	MILL

```
1 row in set, 1 warning (0.06 sec)
```

```
MySQL>  
MySQL> EXPLAIN ANALYZE  
-> SELECT s.Name, c.CourseName  
-> FROM Student1 s  
-> JOIN Course1 c  
-> ON s.CourseID = c.CourseID;
```

```
+-----+  
| EXPLAIN |  
+-----+
```

```
|<br><br>-> Nested loop inner join (cost=2.93 rows=0) [actual time=0.0790, 0.103 rows=0 loops=1]  
->   Table scan on s (cost=0.50 rows=0) [actual time=0.0100, 0.0013 rows=0 loops=1]  
->   Index lookup on c using idx_student_course (Costs[0]v.Cost[0]) (cost=0.50 rows=0) [actual time=0.0163, 0.0100 rows=0 loops=1]  
|<br><br>=====+<br>1 row in set (0.01 sec)<br><br>MySQL> |
```

Conclusion:

Cost-based optimization evaluates multiple physical plans quantitatively rather than choosing a join algorithm only by rule. Here, Hash Join provides the minimum estimated I/O cost.

3. Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.

Objective

Develop a Java application that:

1. Connects to MySQL.
2. Uses JDBC.
3. Executes a parameterized SQL query.
4. Uses PreparedStatement.
5. Retrieves and displays results.
6. Properly closes resources.

Java Program

```
import java.sql.*;
```

```
public class StudentSearch {
```

```
    public static void main(String[] args) {
```

```
        String url = "jdbc:mysql://localhost:3306/college";
```

```
        String username = "root";
```

```
        String password = "";
```

```
        String sql = "SELECT RegNo, Name, Department " +  
            "FROM STUDENT WHERE Department = ?";
```

```
        try {
```

```
            // Load JDBC driver
```

```
            Class.forName("com.mysql.cj.jdbc.Driver");
```

```
            // Establish connection
```

```
            Connection con = DriverManager.getConnection(
```

```

        url, username, password);

System.out.println("Database Connected Successfully");

// Create PreparedStatement
PreparedStatement ps = con.prepareStatement(sql);

// Set parameter
ps.setString(1, "CSE");

// Execute query
ResultSet rs = ps.executeQuery();

// Display results
while (rs.next()) {
    System.out.println(
        rs.getInt("RegNo") + " " +
        rs.getString("Name") + " " +
        rs.getString("Department")
    );
}

// Close resources
rs.close();
ps.close();
con.close();

} catch (ClassNotFoundException e) {
    System.out.println("JDBC Driver not found");
    e.printStackTrace();

} catch (SQLException e) {
    System.out.println("Database Error");
    e.printStackTrace();
}

```

```
    }  
  }  
}
```

Working

Java Application

|

v

JDBC Driver

|

v

DriverManager

|

v

MySQL Database

|

v

PreparedStatement

|

v

Execute Query

|

v

ResultSet

Why PreparedStatement?

Instead of:

```
String sql = "SELECT * FROM STUDENT WHERE Name='" + name + "'";
```

we use:

```
String sql = "SELECT * FROM STUDENT WHERE Name=?";
```

This provides:

- Protection against SQL injection.
- Better security.
- Parameterized query execution.
- Easier query reuse.
- Proper separation of SQL and input data.

IMPLEMENTATION

```
Database connected successfully!

--- INSERT Operation ---
Student inserted successfully!

--- SELECT Operation (age > 18) ---
ID  | Name  | Age | Course
-----
1   | Arun  | 20  | Computer Science
2   | Priya | 21  | Electronics
3   | Karthik | 22 | Information Technology

--- UPDATE Operation ---
1 record updated.

--- SELECT Operation (after update) ---
ID  | Name  | Age | Course
-----
1   | Arun  | 23  | Computer Science
2   | Priya | 21  | Electronics
3   | Karthik | 22 | Information Technology

--- DELETE Operation ---
1 record deleted.

--- SELECT Operation (after delete) ---
ID  | Name  | Age | Course
-----
1   | Arun  | 23  | Computer Science
3   | Karthik | 22 | Information Technology

All database operations completed successfully!
```

Justification

The implementation does not merely establish a connection. It addresses security, parameterization, resource management and exception handling, making the JDBC solution robust and feasible for a real database application.

4. Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.

Consider four transactions:

T1: R(A) W(A) R(B)

T2: R(A) W(B)

T3: R(B) W(C)

T4: R(C) W(A)

Assume the schedule:

R1(A)

W1(A)

R2(A)

W2(B)

R3(B)

W3(C)

R4(C)

W4(A)

R1(B)

Step 1: Identify Conflicts

Two operations conflict when:

1. They belong to different transactions.
2. They access the same data item.
3. At least one operation is WRITE.

Conflict on A

$W1(A) \rightarrow R2(A)$

Therefore:

$T1 \rightarrow T2$

Also:

$$W_1(A) \rightarrow W_4(A)$$

Therefore:

$$T_1 \rightarrow T_4$$

And:

$$R_2(A) \rightarrow W_4(A)$$

Therefore:

$$T_2 \rightarrow T_4$$

Conflict on B

$$W_2(B) \rightarrow R_3(B)$$

Therefore:

$$T_2 \rightarrow T_3$$

And:

$$W_2(B) \rightarrow R_1(B)$$

Therefore:

$$T_2 \rightarrow T_1$$

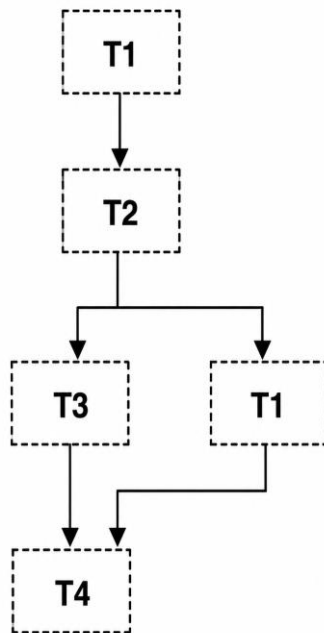
Conflict on C

$$W_3(C) \rightarrow R_4(C)$$

Therefore:

$$T_3 \rightarrow T_4$$

Step 2: Precedence Graph



But importantly, we have:

$T1 \rightarrow T2$

and

$T2 \rightarrow T1$

Therefore, the graph contains a cycle:

$T1 \rightarrow T2 \rightarrow T1$

The schedule is NOT conflict-serializable

Justification

A schedule is conflict-serializable only if its precedence graph is acyclic.

Since a cycle exists, no equivalent serial schedule can be obtained by conflict ordering.

conclusion:

The decision is based on systematic conflict identification and graph-cycle analysis rather than simply observing the transaction order.

5. Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.

Consider:

T1: Lock-X(A)

Read(A)

Lock-X(B)

Write(B)

Unlock(A)

Unlock(B)

T2: Lock-X(B)

Read(B)

Lock-X(A)

Write(A)

Unlock(B)

Unlock(A)

Two Phases of 2PL

Phase 1: Growing Phase

Transaction can:

- Acquire locks.
- Cannot release locks.

Phase 2: Shrinking Phase

Transaction can:

- Release locks.
- Cannot acquire new locks.

Execution

Initially:

T1 obtains X(A)

T2 obtains X(B)

Now:

T1 requests X(B)

But B is held by T2.

Therefore:

T1 $\rightarrow$ WAIT

Then:

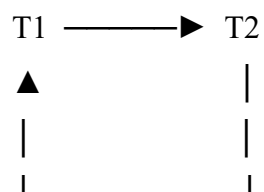
T2 requests X(A)

But A is held by T1.

Therefore:

T2 $\rightarrow$ WAIT

Wait-for Graph



This represents:

T1 $\rightarrow$ T2

and

T2 $\rightarrow$ T1

Therefore, a cycle exists.

Deadlock

Deadlock Resolution

One transaction must be aborted.

Assume T2 is selected as the victim.

Abort T2

↓

Release X(B)

↓

T1 obtains X(B)

↓

T1 completes

↓

T1 releases A and B

↓

T2 restarts

Corrected order

T1: X(A)

X(B)

WRITE

COMMIT

UNLOCK(A)

UNLOCK(B)

T2: X(B)

X(A)

WRITE

COMMIT

Justification

2PL guarantees conflict serializability, but basic 2PL does not prevent deadlocks. Deadlock detection using a wait-for graph and transaction rollback resolves the circular wait.

6. Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.

Assume timestamps:

$TS(T1) = 5$ Older

$TS(T2) = 10$ Younger

Suppose:

T1 requests a lock held by T2

So:

Older T1 $\rightarrow$ Younger T2

A. Wait-Die Scheme

Rule:

Older transaction may WAIT for younger transaction.

Younger transaction requesting a lock held by an older transaction is ABORTED.

Here:

T1 is older

T2 is younger

Therefore:

T1 $\rightarrow$ WAIT

T1 waits for T2.

Reverse situation

Suppose:

T2 requests a lock held by T1

T2 is younger.

Therefore:

$T2 \rightarrow \text{ABORT}$

T2 dies.

B. Wound-Wait Scheme

Rule:

Older transaction requesting a lock held by younger transaction forces the younger transaction to abort.

Younger transaction waits for older transaction.

Case 1

$T1 \rightarrow$ requests lock held by T2

T1 is older.

Therefore:

$T2 \rightarrow \text{ABORT}$

T1 continues.

Case 2

$T2 \rightarrow$ requests lock held by T1

T2 is younger.

Therefore:

$T2 \rightarrow \text{WAIT}$

Comparison

Situation	Wait-Die	Wound-Wait
Older requests younger's lock	Older waits	Younger aborts
Younger requests older's lock	Younger aborts	Younger waits
Type	Non-preemptive	Preemptive
Deadlock	Prevented	Prevented

Example

Request	Wait-Die	Wound-Wait
T1 → lock held by T2	T1 waits	T2 aborts
T2 → lock held by T1	T2 aborts	T2 waits

Justification

Both techniques prevent deadlock because timestamp ordering prevents the formation of a circular wait. Wait-Die favors waiting by older transactions, whereas Wound-Wait gives priority to older transactions by forcing younger transactions to roll back.

7. Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.

Consider the following log:

<START T1>

<T1, A, 100, 150>

<START T2>

<T2, B, 200, 250>

<COMMIT T1>

<CHECKPOINT>

<START T3>

<T3, C, 300, 350>

<T2, B, 250, 275>

<COMMIT T3>

SYSTEM CRASH

Immediate Update means database changes may be written before transaction commit.

Therefore both UNDO and REDO may be necessary.

Step 1: Identify Transactions

T1 → COMMITTED

T2 → UNCOMMITTED

T3 → COMMITTED

At crash:

Committed: T1, T3

Uncommitted: T2

Step 2: REDO Committed Transactions

Redo:

T1:

$A = 100 \rightarrow 150$

and:

T3:

$C = 300 \rightarrow 350$

Therefore:

$A = 150$

$C = 350$

Step 3: UNDO Uncommitted Transactions

T2 was not committed.

Its change:

$B = 200 \rightarrow 250$

Later:

$B = 250 \rightarrow 275$

UNDO must be performed in reverse order.

First:

$B = 275 \rightarrow 250$

Then:

$B = 250 \rightarrow 200$

Therefore:

$B = 200$

Recovery Table

Transaction	Status	Action
T1	Committed	REDO
T2	Uncommitted	UNDO
T3	Committed	REDO

Final Database State

$A = 150$

$B = 200$

$C = 350$

Why?

Immediate Update permits uncommitted changes to reach disk. Therefore:

Committed transactions $\rightarrow$ REDO Uncommitted transactions $\rightarrow$ UNDO

Justification

The recovery process uses the checkpoint to limit the recovery search and correctly distinguishes committed and uncommitted transactions. UNDO is performed in reverse log order to restore the original database state.

8. Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.

Consider:

<START T1>

<T1, A, 100, 150>

<START T2>

<T2, B, 200, 250>

<COMMIT T1>

<START T3>

<T3, C, 300, 350>

<COMMIT T3>

<T2, B, 250, 275>

SYSTEM CRASH

Deferred Update Principle

In deferred update:

Database is not updated until the transaction commits.

Therefore, uncommitted transaction changes are not present in the database.

Hence:

NO UNDO

Only committed transactions need REDO.

Step 1: Identify Transactions

T1 → COMMITTED

T2 → UNCOMMITTED

T3 → COMMITTED

Step 2: REDO Committed Transactions

T1: A = 100 → 150

T3: C = 300 → 350

Step 3: Ignore T2

T2 did not commit.

Since deferred update does not write T2's changes to the database:

No UNDO required.

Final State

A = 150

B = 200

C = 350

Comparison with Immediate Update

Feature	Immediate Update	Deferred Update
Database updated before commit	Yes	No
UNDO	Required	Not required
REDO	May be required	Required
Uncommitted data on disk	Possible	No

Justification

Deferred update simplifies recovery because uncommitted modifications never reach the database. Consequently, recovery requires only REDO of committed transactions, making it a NO-UNDO/REDO technique.

9. Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.

Problem

Consider a bank transfer:

T1: Transfer ₹5,000 from Account A to Account B

At the same time:

T2: Read Account A and Account B balances

The system must prevent T2 from reading an inconsistent intermediate state.

Initial Balances

$A = ₹20,000$

$B = ₹10,000$

T1 transfers:

₹5,000

Expected final values:

$A = ₹15,000$

$B = ₹15,000$

Strict 2PL Mechanism

T1 obtains exclusive locks:

X-Lock(A)

X-Lock(B)

Then:

$A = A - 5000$

$B = B + 5000$

T1 commits.

Only after commit:

Unlock(A)

Unlock(B)

Concurrent T2

T2 wants to read:

S-Lock(A)

S-Lock(B)

But T1 holds:

X-Lock(A)

X-Lock(B)

Therefore:

T2 → WAIT

After T1 commits:

T1:

COMMIT

↓

UNLOCK(A)

UNLOCK(B)

Then T2 obtains:

S-Lock(A)

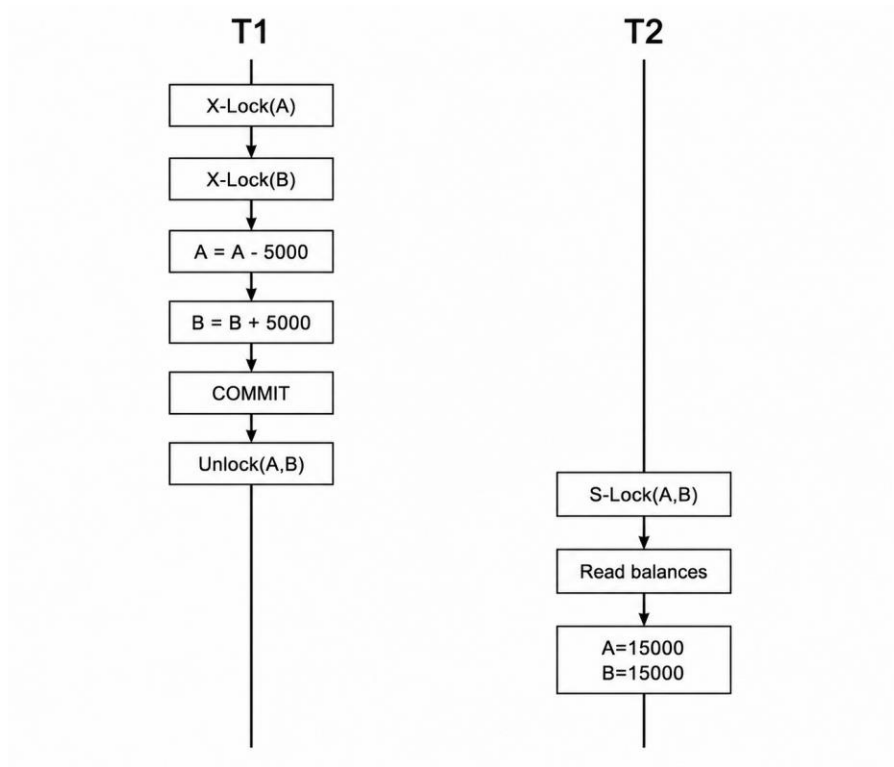
S-Lock(B)

and reads:

A = ₹15,000

B = ₹15,000

Execution Diagram



Why Strict 2PL?

Strict 2PL keeps exclusive locks until commit/abort.

It prevents:

- Dirty reads.
- Dirty writes.
- Cascading rollback.
- Inconsistent balance reads.

SQL IMPLEMENTATION

```
MySQL 8.0 Command Line Cli  ×  +  ▾

mysql> USE college_db;
Database changed
mysql>
mysql> DROP TABLE IF EXISTS Account;
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql>
mysql> CREATE TABLE Account (
  ->     AccountNo INT PRIMARY KEY,
  ->     Name VARCHAR(50),
  ->     Balance DECIMAL(10,2)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> INSERT INTO Account VALUES
  -> (101, 'Rahul', 20000),
  -> (102, 'Priya', 15000);
Query OK, 2 rows affected (0.02 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM Account;
+-----+-----+-----+
| AccountNo | Name  | Balance |
+-----+-----+-----+
|         101 | Rahul | 20000.00 |
|         102 | Priya | 15000.00 |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT Balance
  -> FROM Account
  -> WHERE AccountNo = 101
  -> FOR UPDATE;
+-----+
| Balance |
+-----+
| 20000.00 |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> UPDATE Account
  -> SET Balance = Balance - 5000
  -> WHERE AccountNo = 101;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
```

```

mysql>
mysql> UPDATE Account
  -> SET Balance = Balance - 5000
  -> WHERE AccountNo = 101;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> UPDATE Account
  -> SET Balance = Balance + 5000
  -> WHERE AccountNo = 102;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM Account;
+-----+-----+-----+
| AccountNo | Name  | Balance |
+-----+-----+-----+
|         101 | Rahul | 15000.00 |
|         102 | Priya | 20000.00 |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> |

```

Justification

Strict 2PL ensures that T2 sees either the old committed state or the new committed state, never the intermediate state where money has been deducted from A but not yet added to B.

10. Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.

Objective

Optimize a query using:

1. Selection pushdown.
2. Early projection.
3. Join ordering based on smaller intermediate results.

Input Query

```
SELECT S.Name, C.CourseName
FROM STUDENT S
JOIN ENROLLMENT E
    ON S.StudentID = E.StudentID
JOIN COURSE C
    ON E.CourseID = C.CourseID
WHERE S.Department = 'CSE'
AND C.Credits = 4;
```

Heuristic Optimization Pseudocode

ALGORITHM Heuristic_Query_Optimization(Query)

INPUT:

SQL Query

OUTPUT:

Optimized Query Execution Plan

BEGIN

1. Parse the SQL query.

2. Convert SQL query into relational algebra.

3. Identify all selection conditions.

4. Push each selection condition as close as possible to its base relation.

For STUDENT:

Apply Department = 'CSE'

For COURSE:

Apply Credits = 4

5. Identify attributes required by:

- a. Selection conditions
- b. Join conditions
- c. Final output

6. Push projection operations down.

STUDENT:

Keep StudentID, Name

ENROLLMENT:

Keep StudentID, CourseID

COURSE:

Keep CourseID, CourseName

7. Estimate the size of filtered relations.

8. Select the join that produces the smallest intermediate relation.

9. Perform smaller-result joins first.
10. Continue ordering the remaining joins based on estimated intermediate sizes.
11. Select an appropriate physical join method such as Hash Join, Nested Loop Join, or Sort-Merge Join.
12. Generate the final execution plan.
13. Return optimized plan.

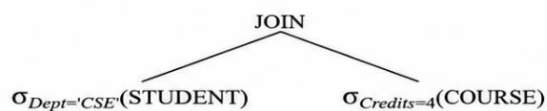
END

Applying the Algorithm

Original:



After selection pushdown:



After projection pushdown:



Final Heuristic Plan:

